

Module 8.3 - Service-Based Communication

Learning Objectives:

- Understand service-based communication patterns
 - Implement shared services for component interaction
 - Master BehaviorSubject for state management
 - Enable communication between unrelated components
 - Apply best practices for service-based data sharing
-

Table of Contents

1. [Introduction to Service-Based Communication](#)
 2. [Shared Services Pattern](#)
 3. [BehaviorSubject for State Management](#)
 4. [Unrelated Components Communication](#)
 5. [Communication Patterns and Best Practices](#)
 6. [Comparison with Other Communication Methods](#)
 7. [Practical Examples](#)
 8. [Summary and Key Takeaways](#)
-

1. Introduction to Service-Based Communication

1.1 What is Service-Based Communication?

Service-based communication is a pattern where components interact by sharing data through a common service rather than directly passing data through parent-child relationships.

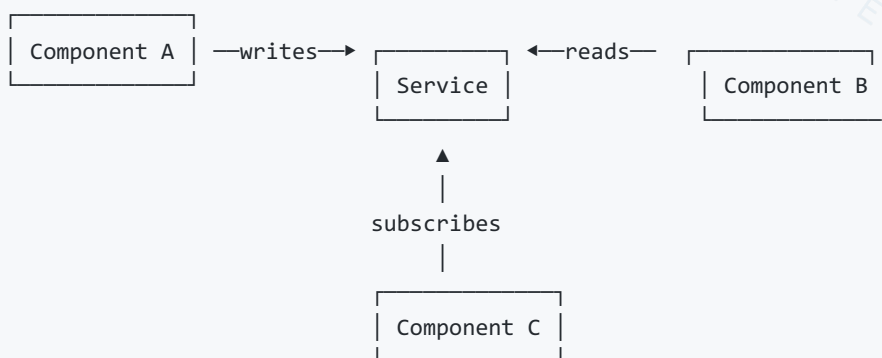
When to Use:

- **Unrelated components:** Components that don't have a parent-child relationship
- **Sibling components:** Components at the same level in the hierarchy
- **Deeply nested components:** Avoiding prop drilling through multiple layers
- **Global state:** Application-wide data that many components need

1.2 Advantages of Service-Based Communication

Advantage	Description
Decoupling	Components don't need to know about each other
Scalability	Easy to add new components that need the same data
Centralization	Single source of truth for shared state
Reusability	Service logic can be reused across components
Testability	Services can be easily mocked for testing

1.3 Communication Flow



2. Shared Services Pattern

2.1 Creating a Shared Service

A shared service acts as a central hub for data and communication between components.

Basic Service Structure:

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root' // Singleton service
})
export class DataSharingService {
  private data: any;

  // Setter method
  setData(value: any): void {
    this.data = value;
  }

  // Getter method
  getData(): any {
    return this.data;
  }
}
```

2.2 Using the Service in Components

Component A (Sender):

```
import { Component } from '@angular/core';
import { DataSharingService } from '../data-sharing.service';

@Component({
  selector: 'app-sender',
  template: `
    <h3>Sender Component</h3>
    <input [(ngModel)]="message" placeholder="Enter message">
    <button (click)="sendMessage()">Send</button>
  `
})
```

```
  })  
  export class SenderComponent {  
    message: string = '';  
  
    constructor(private dataSharingService: DataSharingService) {}  
  
    sendMessage(): void {  
      this.dataSharingService.setData(this.message);  
      console.log('Message sent:', this.message);  
    }  
  }  
}
```

Component B (Receiver):

```
import { Component } from '@angular/core';  
import { DataSharingService } from '../data-sharing.service';  
  
@Component({  
  selector: 'app-receiver',  
  template: `  
    <h3>Receiver Component</h3>  
    <button (click)="receiveMessage()">Get Message</button>  
    <p>Received: {{ receivedMessage }}</p>  
  `,  
})  
export class ReceiverComponent {  
  receivedMessage: string = '';  
  
  constructor(private dataSharingService: DataSharingService) {}  
  
  receiveMessage(): void {  
    this.receivedMessage = this.dataSharingService.getData();  
  }  
}
```

2.3 Limitations of Simple Shared Services

⚠ Problem: The simple getter/setter pattern doesn't provide real-time updates. The receiver must manually call the getter to check for changes.

Solution: Use RxJS Observables for reactive, real-time communication.

3. BehaviorSubject for State Management

3.1 What is BehaviorSubject?

BehaviorSubject is a special type of RxJS Subject that:

- Stores the current value
- Emits the current value to new subscribers immediately
- Allows pushing new values to all subscribers
- Requires an initial value

BehaviorSubject vs Subject vs Observable:

Feature	Observable	Subject	BehaviorSubject
Multicasting	✗ No	☑ Yes	☑ Yes
Push values	✗ No	☑ Yes	☑ Yes
Initial value	✗ N/A	✗ No	☑ Yes (required)
Current value	✗ No	✗ No	☑ Yes (.value)
Late subscribers	Get all from start	Miss previous	Get current value

3.2 Implementing BehaviorSubject

Service with BehaviorSubject:

```
import { Injectable } from '@angular/core';
import { BehaviorSubject, Observable } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class MessageService {
  // Private BehaviorSubject (internal use only)
  private messageSource = new BehaviorSubject<string>('Initial message');

  // Public Observable (for components to subscribe)
```

```
public message$: Observable<string> = this.messageSource.asObservable();

constructor() {}

// Method to update the message
updateMessage(message: string): void {
  this.messageSource.next(message);
}

// Method to get current value (optional)
getCurrentMessage(): string {
  return this.messageSource.value;
}
}
```

💡 **Best Practice:** Expose the BehaviorSubject as an Observable using `.asObservable()` to prevent external code from calling `.next()` directly.

3.3 Subscribing to BehaviorSubject

Component A (Sender):

```
import { Component } from '@angular/core';
import { MessageService } from './message.service';

@Component({
  selector: 'app-message-sender',
  template: `
    <div class="card">
      <h3>Message Sender</h3>
      <input [(ngModel)]="newMessage" placeholder="Type a message">
      <button (click)="send()">Send Message</button>
    </div>
  `
})
export class MessageSenderComponent {
  newMessage: string = '';

  constructor(private messageService: MessageService) {}

  send(): void {
    if (this.newMessage.trim()) {
      this.messageService.updateMessage(this.newMessage);
    }
  }
}
```

```
        this.newMessage = '';  
    }  
}  
}
```

Component B (Receiver):

```
import { Component, OnInit, OnDestroy } from '@angular/core';  
import { Subscription } from 'rxjs';  
import { MessageService } from './message.service';  
  
@Component({  
  selector: 'app-message-receiver',  
  template: `  
    <div class="card">  
      <h3>Message Receiver</h3>  
      <p>Current message: <strong>{{ currentMessage }}</strong></p>  
    </div>  
  `,  
})  
export class MessageReceiverComponent implements OnInit, OnDestroy {  
  currentMessage: string = '';  
  private subscription!: Subscription;  
  
  constructor(private messageService: MessageService) {}  
  
  ngOnInit(): void {  
    // Subscribe to message updates  
    this.subscription = this.messageService.message$.subscribe(  
      message => {  
        this.currentMessage = message;  
        console.log('Received message:', message);  
      }  
    );  
  }  
  
  ngOnDestroy(): void {  
    // Important: Unsubscribe to prevent memory leaks  
    if (this.subscription) {  
      this.subscription.unsubscribe();  
    }  
  }  
}
```

3.4 Using Async Pipe (Automatic Subscription Management)

The `async` pipe automatically subscribes and unsubscribes, preventing memory leaks.

```
import { Component, OnInit } from '@angular/core';
import { Observable } from 'rxjs';
import { MessageService } from './message.service';

@Component({
  selector: 'app-message-receiver',
  template: `
    <div class="card">
      <h3>Message Receiver</h3>
      <p>Current message: <strong>{{ message$ | async }}</strong></p>
    </div>
  `
})
export class MessageReceiverComponent implements OnInit {
  message$: Observable<string>;

  constructor(private messageService: MessageService) {}

  ngOnInit(): void {
    this.message$ = this.messageService.message$;
  }
}
```

☒ **Benefit:** No need to manually unsubscribe in `ngOnDestroy` !

3.5 Complex State Management

Managing User State:

```
import { Injectable } from '@angular/core';
import { BehaviorSubject, Observable } from 'rxjs';

export interface User {
  id: number;
  name: string;
  email: string;
  isLoggedIn: boolean;
}
```

```
@Injectable({
  providedIn: 'root'
})
export class UserStateService {
  private initialUser: User = {
    id: 0,
    name: '',
    email: '',
    isLoggedIn: false
  };

  private userSubject = new BehaviorSubject<User>(this.initialUser);
  public user$: Observable<User> = this.userSubject.asObservable();

  constructor() {}

  // Update entire user object
  setUser(user: User): void {
    this.userSubject.next(user);
  }

  // Update specific property
  updateUserName(name: string): void {
    const currentUser = this.userSubject.value;
    this.userSubject.next({ ...currentUser, name });
  }

  // Login
  login(user: User): void {
    this.userSubject.next({ ...user, isLoggedIn: true });
  }

  // Logout
  logout(): void {
    this.userSubject.next(this.initialUser);
  }

  // Get current user synchronously
  getCurrentUser(): User {
    return this.userSubject.value;
  }
}
```

Using in Component:

```
import { Component, OnInit } from '@angular/core';
import { Observable } from 'rxjs';
import { User, UserStateService } from '../user-state.service';

@Component({
  selector: 'app-user-profile',
  template: `
    <div *ngIf="user$ | async as user">
      <h3>User Profile</h3>
      <div *ngIf="user.isLoggedIn; else notLoggedIn">
        <p>Name: {{ user.name }}</p>
        <p>Email: {{ user.email }}</p>
        <button (click)="logout()">Logout</button>
      </div>
      <ng-template #notLoggedIn>
        <p>Please log in</p>
      </ng-template>
    </div>
  `,
})
export class UserProfileComponent implements OnInit {
  user$: Observable<User>;

  constructor(private userStateService: UserStateService) {}

  ngOnInit(): void {
    this.user$ = this.userStateService.user$;
  }

  logout(): void {
    this.userStateService.logout();
  }
}
```

4. Unrelated Components Communication

4.1 Communication Between Sibling Components

Sibling components (components at the same level) can communicate through a shared service without involving the parent.

Scenario: Shopping cart where product list and cart summary are siblings.

```
// cart.service.ts
import { Injectable } from '@angular/core';
import { BehaviorSubject, Observable } from 'rxjs';

export interface CartItem {
  id: number;
  name: string;
  price: number;
  quantity: number;
}

@Injectable({
  providedIn: 'root'
})
export class CartService {
  private cartItems = new BehaviorSubject<CartItem[]>([]);
  public cartItems$: Observable<CartItem[]> = this.cartItems.asObservable();

  constructor() {}

  addToCart(item: CartItem): void {
    const currentItems = this.cartItems.value;
    const existingItem = currentItems.find(i => i.id === item.id);

    if (existingItem) {
      existingItem.quantity += item.quantity;
      this.cartItems.next([...currentItems]);
    } else {
      this.cartItems.next([...currentItems, item]);
    }
  }

  removeFromCart(itemId: number): void {
    const currentItems = this.cartItems.value;
    const updatedItems = currentItems.filter(item => item.id !== itemId);
    this.cartItems.next(updatedItems);
  }

  clearCart(): void {
    this.cartItems.next([]);
  }

  getTotalPrice(): number {
    return this.cartItems.value.reduce(
```

```

    (total, item) => total + (item.price * item.quantity),
    0
  );
}
}

```

Product List Component (Sibling 1):

```

import { Component } from '@angular/core';
import { CartService } from './cart.service';

@Component({
  selector: 'app-product-list',
  template: `
    <div class="products">
      <h3>Products</h3>
      <div *ngFor="let product of products" class="product-card">
        <h4>{{ product.name }}</h4>
        <p>Price: \${{ product.price }}</p>
        <button (click)="addToCart(product)">Add to Cart</button>
      </div>
    </div>
  `
})
export class ProductListComponent {
  products = [
    { id: 1, name: 'Laptop', price: 999 },
    { id: 2, name: 'Mouse', price: 29 },
    { id: 3, name: 'Keyboard', price: 79 }
  ];

  constructor(private cartService: CartService) {}

  addToCart(product: any): void {
    this.cartService.addToCart({ ...product, quantity: 1 });
  }
}

```

Cart Summary Component (Sibling 2):

```

import { Component, OnInit } from '@angular/core';
import { Observable } from 'rxjs';
import { map } from 'rxjs/operators';

```

```
import { CartService, CartItem } from './cart.service';

@Component({
  selector: 'app-cart-summary',
  template: `
    <div class="cart">
      <h3>Shopping Cart</h3>
      <div *ngIf="(cartItems$ | async) as items">
        <div *ngIf="items.length > 0; else emptyCart">
          <div *ngFor="let item of items" class="cart-item">
            <span>{{ item.name }} (x{{ item.quantity }})</span>
            <span>\${{ item.price * item.quantity }}</span>
            <button (click)="remove(item.id)">Remove</button>
          </div>
        <hr>
        <p><strong>Total: \${{ totalPrice$ | async }}</strong></p>
        <button (click)="clearCart()">Clear Cart</button>
      </div>
      <ng-template #emptyCart>
        <p>Your cart is empty</p>
      </ng-template>
    </div>
  `
})
export class CartSummaryComponent implements OnInit {
  cartItems$: Observable<CartItem[]>;
  totalPrice$: Observable<number>;

  constructor(private cartService: CartService) {}

  ngOnInit(): void {
    this.cartItems$ = this.cartService.cartItems$;

    // Calculate total using RxJS operators
    this.totalPrice$ = this.cartItems$.pipe(
      map(items => items.reduce((total, item) =>
        total + (item.price * item.quantity), 0
      ))
    );
  }

  remove(itemId: number): void {
    this.cartService.removeFromCart(itemId);
  }

  clearCart(): void {

```

```
    this.cartService.clearCart();  
  }  
}
```

4.2 Communication Across Distant Components

Components in different parts of the component tree can communicate through a shared service.

```
App Component  
├─ Header Component (shows notification count)  
├─ Sidebar Component  
└─ Main Content  
    └─ Notification Panel (sends notifications)
```

Notification Service:

```
import { Injectable } from '@angular/core';  
import { BehaviorSubject, Observable } from 'rxjs';  
  
export interface Notification {  
  id: number;  
  message: string;  
  type: 'info' | 'warning' | 'error' | 'success';  
  timestamp: Date;  
}  
  
@Injectable({  
  providedIn: 'root'  
})  
export class NotificationService {  
  private notificationsSubject = new BehaviorSubject<Notification[]>([]);  
  public notifications$: Observable<Notification[]> =  
    this.notificationsSubject.asObservable();  
  
  private idCounter = 0;  
  
  addNotification(message: string, type: Notification['type'] = 'info'): void {  
    const notification: Notification = {  
      id: ++this.idCounter,  
      message,  
      type,  
      timestamp: new Date()  
    }  
  }  
}
```

```

    };

    const current = this.notificationsSubject.value;
    this.notificationsSubject.next([notification, ...current]);
  }

  removeNotification(id: number): void {
    const current = this.notificationsSubject.value;
    const updated = current.filter(n => n.id !== id);
    this.notificationsSubject.next(updated);
  }

  clearAll(): void {
    this.notificationsSubject.next([]);
  }

  getCount(): number {
    return this.notificationsSubject.value.length;
  }
}

```

Header Component (displays count):

```

import { Component, OnInit } from '@angular/core';
import { Observable } from 'rxjs';
import { map } from 'rxjs/operators';
import { NotificationService } from './notification.service';

@Component({
  selector: 'app-header',
  template: `
    <header>
      <h1>My App</h1>
      <div class="notification-badge">
        🔔 <span class="badge">{{ notificationCount$ | async }}</span>
      </div>
    </header>
  `
})
export class HeaderComponent implements OnInit {
  notificationCount$: Observable<number>;

  constructor(private notificationService: NotificationService) {}

  ngOnInit(): void {

```

```
this.notificationCount$ = this.notificationService.notifications$.pipe(  
  map(notifications => notifications.length)  
);  
}  
}
```

Content Component (creates notifications):

```
import { Component } from '@angular/core';  
import { NotificationService } from '../notification.service';  
  
@Component({  
  selector: 'app-content',  
  template: `  
    <div>  
      <h2>Main Content</h2>  
      <button (click)="notify('info')">Info Notification</button>  
      <button (click)="notify('success')">Success Notification</button>  
      <button (click)="notify('warning')">Warning Notification</button>  
      <button (click)="notify('error')">Error Notification</button>  
    </div>  
  `,  
})  
export class ContentComponent {  
  constructor(private notificationService: NotificationService) {}  
  
  notify(type: 'info' | 'success' | 'warning' | 'error'): void {  
    const messages = {  
      info: 'This is an information message',  
      success: 'Operation completed successfully!',  
      warning: 'This is a warning',  
      error: 'An error occurred!'  
    };  
  
    this.notificationService.addNotification(messages[type], type);  
  }  
}
```

5. Communication Patterns and Best Practices

5.1 Common Patterns

Pattern	Use Case	Implementation
Message Bus	Global event system	Service with Subject/ EventEmitter
State Store	Centralized state	Service with BehaviorSubject
Observable Data Service	Real-time data sync	Service exposing Observables
Command Pattern	Execute actions	Service with action methods

5.2 Best Practices

1. Use BehaviorSubject for State

```
// ☒ Good: BehaviorSubject with initial state
private userSubject = new BehaviorSubject<User | null>(null);
public user$ = this.userSubject.asObservable();

// ☒ Avoid: Regular Subject (new subscribers miss current value)
private userSubject = new Subject<User>();
```

2. Expose Observable, Hide Subject

```
// ☒ Good: Only expose Observable
private dataSubject = new BehaviorSubject<string>('');
public data$ = this.dataSubject.asObservable();

// ☒ Avoid: Exposing Subject directly allows external .next() calls
public dataSubject = new BehaviorSubject<string>('');
```

3. Always Unsubscribe

```
// ☒ Option 1: Manual unsubscribe
ngOnDestroy() {
  this.subscription.unsubscribe();
}

// ☒ Option 2: Use async pipe (auto-unsubscribe)
template: `{{ data$ | async }}`

// ☒ Option 3: takeUntil pattern
import { Subject } from 'rxjs';
import { takeUntil } from 'rxjs/operators';

private destroy$ = new Subject<void>();

ngOnInit() {
  this.service.data$
    .pipe(takeUntil(this.destroy$))
    .subscribe(data => this.data = data);
}

ngOnDestroy() {
  this.destroy$.next();
  this.destroy$.complete();
}
```

4. Immutable Updates

```
// ☒ Good: Create new object
updateUser(name: string) {
  const current = this.userSubject.value;
  this.userSubject.next({ ...current, name });
}

// ☒ Avoid: Mutating object
updateUser(name: string) {
  const current = this.userSubject.value;
  current.name = name; // Direct mutation
  this.userSubject.next(current);
}
```

5. Type Safety

```
// ☒ Good: Strongly typed
private dataSubject = new BehaviorSubject<string>('');

// ☒ Avoid: Using 'any'
private dataSubject = new BehaviorSubject<any>(null);
```

6. Service Naming Conventions

```
// Observable properties end with $
public users$: Observable<User[]>;
public currentUser$: Observable<User | null>;

// Methods are verbs
setCurrentUser(user: User): void { }
updateUserSettings(settings: UserSettings): void { }
```

5.3 Memory Leak Prevention

Common Memory Leak Scenario:

```
// ☒ Memory leak: Not unsubscribing
ngOnInit() {
  this.service.data$.subscribe(data => {
    this.data = data;
  });
  // When component is destroyed, subscription remains active!
}
```

Solutions:

```
// ☒ Solution 1: Store and unsubscribe
private subscription!: Subscription;

ngOnInit() {
  this.subscription = this.service.data$.subscribe(data => {
    this.data = data;
  });
}
```

```

ngOnDestroy() {
  this.subscription.unsubscribe();
}

// ☒ Solution 2: Multiple subscriptions with Subscription container
private subscriptions = new Subscription();

ngOnInit() {
  this.subscriptions.add(
    this.service.data1$.subscribe(data => this.data1 = data)
  );
  this.subscriptions.add(
    this.service.data2$.subscribe(data => this.data2 = data)
  );
}

ngOnDestroy() {
  this.subscriptions.unsubscribe(); // Unsubscribes all
}

// ☒ Solution 3: Use async pipe (recommended)
// No manual subscription needed
template: `

{{ data$ | async }}

`

```

6. Comparison with Other Communication Methods

6.1 When to Use Which Method

Scenario	Method	Reason
Parent → Child	@Input	Simple, direct, type-safe
Child → Parent	@Output + EventEmitter	Clear event contract
Sibling Components	Shared Service	No common parent needed
Unrelated Components	Shared Service	Decoupled communication
Global State	Service + BehaviorSubject	Single source of truth
Complex State	NgRx/Akita (State Mgmt)	Advanced patterns needed

6.2 Detailed Comparison

Feature	@Input/@Output	ViewChild	Service-Based
Relationship	Parent-Child only	Parent-Child only	Any components
Coupling	Tightly coupled	Very tightly coupled	Loosely coupled
Scalability	Limited (prop drilling)	Limited	Highly scalable
Real-time Updates	Yes	Yes	Yes (with Observable)
Testability	Good	Moderate	Excellent
Setup Complexity	Simple	Simple	Moderate
State Persistence	No	No	Yes

6.3 Decision Tree

```

Are components parent-child?
├─ YES
│   ├── Parent to Child? → Use @Input
│   ├── Child to Parent? → Use @Output
│   └─ Need direct access? → Use ViewChild
└─ NO (Siblings or Unrelated)
    ├── Simple one-time data? → Service with getter/setter
    ├── Real-time updates needed? → Service with BehaviorSubject
    ├── Complex state management? → Consider NgRx/Akita
    └─ Global configuration? → Use InjectionToken
  
```

7. Practical Examples

7.1 Example 1: Theme Switcher (Global State)

theme.service.ts:

```
import { Injectable } from '@angular/core';
import { BehaviorSubject, Observable } from 'rxjs';

export type Theme = 'light' | 'dark' | 'auto';

@Injectable({
  providedIn: 'root'
})
export class ThemeService {
  private readonly THEME_KEY = 'app-theme';

  private themeSubject: BehaviorSubject<Theme>;
  public theme$: Observable<Theme>;

  constructor() {
    // Load theme from localStorage or default to 'light'
    const savedTheme = (localStorage.getItem(this.THEME_KEY) as Theme) || 'light';
    this.themeSubject = new BehaviorSubject<Theme>(savedTheme);
    this.theme$ = this.themeSubject.asObservable();

    // Apply theme on initialization
    this.applyTheme(savedTheme);
  }

  setTheme(theme: Theme): void {
    this.themeSubject.next(theme);
    localStorage.setItem(this.THEME_KEY, theme);
    this.applyTheme(theme);
  }

  getCurrentTheme(): Theme {
    return this.themeSubject.value;
  }

  toggleTheme(): void {
    const current = this.getCurrentTheme();
    const next: Theme = current === 'light' ? 'dark' : 'light';
    this.setTheme(next);
  }

  private applyTheme(theme: Theme): void {
    document.body.className = theme === 'dark' ? 'dark-theme' : 'light-theme';
  }
}
```

theme-switcher.component.ts:

```
import { Component, OnInit } from '@angular/core';
import { Observable } from 'rxjs';
import { Theme, ThemeService } from '../theme.service';

@Component({
  selector: 'app-theme-switcher',
  template: `
    <div class="theme-switcher">
      <label>Theme:</label>
      <select [value]="theme$ | async" (change)="onThemeChange($event)">
        <option value="light">Light</option>
        <option value="dark">Dark</option>
        <option value="auto">Auto</option>
      </select>
      <button (click)="toggle()">Toggle Theme</button>
    </div>
  `,
  styles: [`
    .theme-switcher {
      padding: 10px;
      display: flex;
      gap: 10px;
      align-items: center;
    }
  `]
})
export class ThemeSwitcherComponent implements OnInit {
  theme$: Observable<Theme>;

  constructor(private themeService: ThemeService) {}

  ngOnInit(): void {
    this.theme$ = this.themeService.theme$;
  }

  onThemeChange(event: Event): void {
    const theme = (event.target as HTMLSelectElement).value as Theme;
    this.themeService.setTheme(theme);
  }

  toggle(): void {

```

```
    this.themeService.toggleTheme();  
  }  
}
```

7.2 Example 2: Search Filter (Multi-Component Sync)

search.service.ts:

```
import { Injectable } from '@angular/core';  
import { BehaviorSubject, Observable, combineLatest } from 'rxjs';  
import { map, debounceTime, distinctUntilChanged } from 'rxjs/operators';  
  
export interface SearchFilters {  
  searchTerm: string;  
  category: string;  
  minPrice: number;  
  maxPrice: number;  
}  
  
@Injectable({  
  providedIn: 'root'  
})  
export class SearchService {  
  private searchTermSubject = new BehaviorSubject<string>('');  
  private categorySubject = new BehaviorSubject<string>('all');  
  private minPriceSubject = new BehaviorSubject<number>(0);  
  private maxPriceSubject = new BehaviorSubject<number>(10000);  
  
  // Individual observables  
  public searchTerm$ = this.searchTermSubject.asObservable().pipe(  
    debounceTime(300),  
    distinctUntilChanged()  
  );  
  public category$ = this.categorySubject.asObservable();  
  public minPrice$ = this.minPriceSubject.asObservable();  
  public maxPrice$ = this.maxPriceSubject.asObservable();  
  
  // Combined filters observable  
  public filters$: Observable<SearchFilters> = combineLatest([  
    this.searchTerm$,  
    this.category$,  
    this.minPrice$,  
    this.maxPrice$  
  ]).pipe(  

```

```

    map([searchTerm, category, minPrice, maxPrice]) => ({
      searchTerm,
      category,
      minPrice,
      maxPrice
    })))
  );

  setSearchTerm(term: string): void {
    this.searchTermSubject.next(term);
  }

  setCategory(category: string): void {
    this.categorySubject.next(category);
  }

  setPriceRange(min: number, max: number): void {
    this.minPriceSubject.next(min);
    this.maxPriceSubject.next(max);
  }

  resetFilters(): void {
    this.searchTermSubject.next('');
    this.categorySubject.next('all');
    this.minPriceSubject.next(0);
    this.maxPriceSubject.next(10000);
  }
}

```

search-bar.component.ts:

```

import { Component } from '@angular/core';
import { SearchService } from '../search.service';

@Component({
  selector: 'app-search-bar',
  template: `
    <div class="search-bar">
      <input
        type="text"
        placeholder="Search products..."
        (input)="onSearchChange($event)"
      >
    </div>
  `
})

```

```

  })
  export class SearchBarComponent {
    constructor(private searchService: SearchService) {}

    onSearchChange(event: Event): void {
      const term = (event.target as HTMLInputElement).value;
      this.searchService.setSearchTerm(term);
    }
  }
}

```

product-list.component.ts:

```

import { Component, OnInit } from '@angular/core';
import { Observable } from 'rxjs';
import { map } from 'rxjs/operators';
import { SearchService, SearchFilters } from '../search.service';

interface Product {
  id: number;
  name: string;
  category: string;
  price: number;
}

@Component({
  selector: 'app-product-list',
  template: `
    <div class="product-list">
      <h3>Products</h3>
      <div *ngFor="let product of filteredProducts$ | async" class="product">
        <h4>{{ product.name }}</h4>
        <p>Category: {{ product.category }}</p>
        <p>Price: \${{ product.price }}</p>
      </div>
    </div>
  `,
})
export class ProductListComponent implements OnInit {
  filteredProducts$: Observable<Product[]>;

  products: Product[] = [
    { id: 1, name: 'Laptop', category: 'electronics', price: 999 },
    { id: 2, name: 'Phone', category: 'electronics', price: 699 },
    { id: 3, name: 'Desk', category: 'furniture', price: 299 },
    { id: 4, name: 'Chair', category: 'furniture', price: 199 }
  ]
}

```

```
];

constructor(private searchService: SearchService) {}

ngOnInit(): void {
  this.filteredProducts$ = this.searchService.filters$.pipe(
    map(filters => this.filterProducts(filters))
  );
}

private filterProducts(filters: SearchFilters): Product[] {
  return this.products.filter(product => {
    const matchesSearch = product.name.toLowerCase()
      .includes(filters.searchTerm.toLowerCase());
    const matchesCategory = filters.category === 'all' ||
      product.category === filters.category;
    const matchesPrice = product.price >= filters.minPrice &&
      product.price <= filters.maxPrice;

    return matchesSearch && matchesCategory && matchesPrice;
  });
}
```

7.3 Example 3: Form Wizard (Multi-Step State)

wizard.service.ts:

```
import { Injectable } from '@angular/core';
import { BehaviorSubject, Observable } from 'rxjs';

export interface WizardState {
  currentStep: number;
  totalSteps: number;
  data: { [key: string]: any };
  isComplete: boolean;
}

@Injectable({
  providedIn: 'root'
})
export class WizardService {
  private initialState: WizardState = {
    currentStep: 1,
```

```
totalSteps: 3,  
data: {},  
isComplete: false  
};  
  
private stateSubject = new BehaviorSubject<WizardState>(this.initialState);  
public state$: Observable<WizardState> = this.stateSubject.asObservable();  
  
nextStep(): void {  
  const current = this.stateSubject.value;  
  if (current.currentStep < current.totalSteps) {  
    this.stateSubject.next({  
      ...current,  
      currentStep: current.currentStep + 1  
    });  
  }  
}  
  
previousStep(): void {  
  const current = this.stateSubject.value;  
  if (current.currentStep > 1) {  
    this.stateSubject.next({  
      ...current,  
      currentStep: current.currentStep - 1  
    });  
  }  
}  
  
goToStep(step: number): void {  
  const current = this.stateSubject.value;  
  if (step >= 1 && step <= current.totalSteps) {  
    this.stateSubject.next({  
      ...current,  
      currentStep: step  
    });  
  }  
}  
  
updateStepData(stepData: { [key: string]: any }): void {  
  const current = this.stateSubject.value;  
  this.stateSubject.next({  
    ...current,  
    data: { ...current.data, ...stepData }  
  });  
}  
  
completeWizard(): void {
```

```
const current = this.stateSubject.value;
this.stateSubject.next({
  ...current,
  isComplete: true
});
}

reset(): void {
  this.stateSubject.next(this.initialState);
}

getWizardData(): { [key: string]: any } {
  return this.stateSubject.value.data;
}
}
```

8. Summary and Key Takeaways

8.1 Key Concepts

1. Service-Based Communication

- Enables communication between unrelated components
- Uses services as a central hub for data sharing
- Decouples components for better scalability

2. BehaviorSubject

- Stores current state value
- Immediately emits to new subscribers
- Ideal for state management
- Requires initial value

3. Observable Pattern

- Reactive, real-time updates
- Multiple subscribers can receive updates

- Must unsubscribe to prevent memory leaks

4. Best Practices

- Expose Observable, hide Subject
- Always unsubscribe (or use async pipe)
- Use immutable updates
- Maintain type safety

8.2 When to Use Service-Based Communication

☒ Use When:

- Components are not parent-child
- Need to share state across multiple components
- Want to decouple components
- Need reactive, real-time updates
- Managing global application state

☒ Avoid When:

- Simple parent-child communication (use @Input/@Output)
- One-time data passing
- Very complex state (consider NgRx/Akita)

8.3 Common Pitfalls

Pitfall	Problem	Solution
Not unsubscribing	Memory leaks	Use async pipe or unsubscribe in ngOnDestroy
Exposing Subject	External code can emit values	Expose as Observable with .asObservable()
Mutating state	Change detection issues	Always create new objects

Pitfall	Problem	Solution
Using 'any' type	Loss of type safety	Define proper interfaces/types
Too many services	Over-engineering	Group related state in single service

8.4 Quick Reference

Creating a Service:

```
@Injectable({ providedIn: 'root' })
export class DataService {
  private dataSubject = new BehaviorSubject<T>(initialValue);
  public data$ = this.dataSubject.asObservable();

  updateData(value: T): void {
    this.dataSubject.next(value);
  }
}
```

Subscribing in Component:

```
// Option 1: Async pipe (recommended)
data$ = this.service.data$;
template: `{{ data$ | async }}`

// Option 2: Manual subscription
this.subscription = this.service.data$.subscribe(data => {
  this.data = data;
});
// Remember to unsubscribe in ngOnDestroy!
```

8.5 Next Steps

- Practice building services with BehaviorSubject
- Implement cross-component communication patterns
- Explore RxJS operators for advanced transformations

- Learn about NgRx for complex state management
- Study reactive programming patterns

Additional Resources

- **RxJS Documentation:** <https://rxjs.dev/>
- **Angular Services:** <https://angular.io/guide/architecture-services>
- **Dependency Injection:** <https://angular.io/guide/dependency-injection>
- **BehaviorSubject:** <https://rxjs.dev/api/index/class/BehaviorSubject>

💡 **Pro Tip:** Start with simple getter/setter services, then graduate to BehaviorSubject when you need reactive updates. Don't over-engineer—choose the simplest solution that meets your needs!